

```

#include <iostream>
#include <signal.h>
#include <unistd.h>
#include <fcntl.h>
#include <sstream>
#include <map>
#include <chrono>
#include <pigpio.h>
#include <string>

#include <sys/ioctl.h>
#include <asm/termbits.h>

#include "sl_lidar.h"
#include "sl_lidar_driver.h"

using namespace sl;
ILidarDriver* lidar = nullptr;

// -----
// millis() Linux
// -----
unsigned long millis() {
    using namespace std::chrono;
    static steady_clock::time_point start = steady_clock::now();
    return (unsigned long)duration_cast<std::chrono::milliseconds>(
        std::chrono::steady_clock::now() - start
    ).count();
}

// -----
// Serial config
// -----
void configSerialPort(int fd, int baudrate) {
    struct termios2 tio;

    ioctl(fd, TCGETS2, &tio);

    tio.c_cflag &= ~CBAUD;
    tio.c_cflag |= BOTHER;
    tio.c_ispeed = baudrate;
    tio.c_ospeed = baudrate;

    tio.c_cflag &= ~CRTSCTS;
    tio.c_cflag |= CREAD | CLOCAL;
    tio.c_cflag &= ~PARENB;
    tio.c_cflag &= ~CSTOPB;
    tio.c_cflag &= ~CSIZE;

```

```

    tio.c_cflag |= CS8;

    tio.c_cc[VTIME] = 0;
    tio.c_cc[VMIN] = 0;

    ioctl(fd, TCSETS2, &tio);
}

// -----
// Read some bytes (FIXED)
// -----
std::string serialReadSome(int fd) {
    char buf[256];
    int n = read(fd, buf, sizeof(buf));
    if (n > 0) {
        return std::string(buf, n); // keep all bytes
    }
    return "";
}

// -----
// Ctrl+C handler
// -----
void ctrlc_handler(int) {
    if (lidar) {
        lidar->stop();
        lidar->reset();
        delete lidar;
        lidar = nullptr;
    }
    gpioTerminate();
    exit(0);
}

// =====
// GROVE ULTRASONIC
// =====
float readGroveUltrasonic(int pin)
{
    gpioSetMode(pin, PI_OUTPUT);
    gpioWrite(pin, 0);
    gpioDelay(2);

    gpioWrite(pin, 1);
    gpioDelay(10);
    gpioWrite(pin, 0);

    gpioSetMode(pin, PI_INPUT);

```

```

uint32_t start_wait = gpioTick();
while (gpioRead(pin) == 0) {
    if (gpioTick() - start_wait > 30000) return -1;
}

uint32_t high_start = gpioTick();
while (gpioRead(pin) == 1) {
    if (gpioTick() - high_start > 30000) return -1;
}
uint32_t high_end = gpioTick();

uint32_t duration = high_end - high_start;
return duration / 58.0f;
}

// =====
// MAIN
// =====
int main() {
    signal(SIGINT, ctrlc_handler);
    int status = 0;

    if (gpiolInitialise() < 0) {
        std::cerr << "pigpio init failed!" << std::endl;
        return 1;
    }

    // -----
    // XBee init
    // -----
    int xbee_fd = open("/dev/ttyUSB2", O_RDWR | O_NOCTTY | O_NDELAY);
    if (xbee_fd < 0) {
        std::cerr << "[XBEE] Impossible d'ouvrir /dev/ttyUSB2" << std::endl;
        return 1;
    }
    fcntl(xbee_fd, F_SETFL, 0);
    configSerialPort(xbee_fd, 9600);

    std::string xbeeBuffer;

    // Attente START
    std::cout << "[XBEE] En attente du signal START..." << std::endl;
    while (status == 0) {
        std::string chunk = serialReadSome(xbee_fd);

        if (!chunk.empty()) {
            xbeeBuffer += chunk;

```

```

        if (xbeeBuffer.find("START") != std::string::npos) {
            std::cout << "[XBEE] START reçu → démarrage du robot !" << std::endl;
            status = 1;
            xbeeBuffer.clear();
        }
    }
    usleep(10000);
}

// -----
// LIDAR init
// -----
auto channel_res = createSerialPortChannel("/dev/ttyUSB0", 256000);
IChannel* channel = *channel_res;

auto driver_res = createLidarDriver();
lidar = *driver_res;

lidar->connect(channel);
lidar->reset(5000);
usleep(3000000);

lidar->setMotorSpeed();
lidar->startScan(false, false);
lidar->setMotorSpeed(200);

sl_lidar_response_measurement_node_hq_t nodes[8192];
size_t count = sizeof(nodes)/sizeof(nodes[0]);

const float FLOOR_OBSTACLE_THRESHOLD_CM = 30.0f;

// =====
// MAIN LOOP
// =====
while (status == 1) {

    // LIDAR processing
    float sum_dist[360] = {0};
    int count_dist[360] = {0};

    count = sizeof(nodes)/sizeof(nodes[0]);
    lidar->grabScanDataHq(nodes, count, 0);
    lidar->ascendScanData(nodes, count);

    float avg0 = 0;
    float maxValue = 0;
    int maxAngle = 0;

```

```

    for (size_t i = 0; i < count; ++i) {
        int angle = (nodes[i].angle_z_q14 * 90) >> 14;
        float dist = nodes[i].dist_mm_q2 / 4.0f;

        if ((angle >= 270 && angle < 360) || (angle >= 0 && angle <= 90)) {
            sum_dist[angle] += dist;
            count_dist[angle]++;
        }
    }

    for (int a = 0; a < 360; ++a) {
        if (count_dist[a] > 0) {
            float avg = sum_dist[a] / count_dist[a];
            if (a == 0) avg0 = avg;
            if (avg > maxValue) {
                maxValue = avg;
                maxAngle = a;
            }
        }
    }

    int sendAngle = (avg0 > 500) ? 0 : maxAngle;
    int speed = 1680;

    if (avg0 > 500) speed = 1680;
    else if (avg0 >= 300) speed = 1650;
    else if (avg0 >= 200) speed = 1630;
    else if (avg0 >= 100) speed = 1620;
    else {
        speed = 1350;
        sendAngle = (sendAngle + 180) % 360;
    }

    // Ultrason sol
    float floorDist = readGroveUltrasonic(22);
    bool floorObstacle = (floorDist > 0 && floorDist <
FLOOR_OBSTACLE_THRESHOLD_CM);

    if (floorObstacle) {
        std::cout << "[ULTRA] obstacle sol à " << floorDist << " cm";
        if (sendAngle == 0) {
            if (maxAngle >= 270 && maxAngle < 360) {
                sendAngle = 270;
            }
            else if (maxAngle >= 0 && maxAngle <= 90) {
                sendAngle = 90;
            }
        }
    }

```

```

    }
    std::cout << " | angle=" << sendAngle << std::endl;
    speed = 1300;
} else {
    std::cout << "[LIDAR] dist_front=" << avg0
        << " | angle=" << sendAngle
        << " | speed=" << speed
        << (speed < 1500 ? " (REVERSE)" : " (FORWARD)")
        << std::endl;
}

// Send JSON to motor controller
std::stringstream ss;
ss << "{\"action\":\"drive\", \"angle\": " << sendAngle
    << ", \"speed\": " << speed << "}";

int serial_port = open("/dev/ttyUSB1", O_RDWR | O_NOCTTY | O_NDELAY);
if (serial_port >= 0) {
    fcntl(serial_port, F_SETFL, 0);
    configSerialPort(serial_port, 115200);
    std::string msg = ss.str() + "\n";
    write(serial_port, msg.c_str(), msg.size());
    close(serial_port);
}

// -----
// READ XBEE + DEBUG RAW DATA
// -----
std::string chunk = serialReadSome(xbee_fd);
if (!chunk.empty()) {

    // DEBUG RAW BYTES
    std::cout << "[XBEE RAW] " << chunk.size() << " bytes: ";
    for (unsigned char c : chunk) {
        std::cout << std::hex << (int)c << " ";
    }
    std::cout << std::dec << std::endl;

    xbeeBuffer += chunk;

    if (xbeeBuffer.find("END") != std::string::npos) {
        std::cout << "[XBEE] END reçu → arrêt immédiat du robot !" << std::endl;

        int serial_port = open("/dev/ttyUSB1", O_RDWR | O_NOCTTY | O_NDELAY);
        if (serial_port >= 0) {
            fcntl(serial_port, F_SETFL, 0);
            configSerialPort(serial_port, 115200);
            std::string stopMsg = "{\"action\":\"drive\", \"angle\":0, \"speed\":1500}\n";

```

```
        write(serial_port, stopMsg.c_str(), stopMsg.size());  
        close(serial_port);  
    }
```

```
        lidar->stop();  
        lidar->reset();  
        delete lidar;  
        lidar = nullptr;
```

```
        status = 0;  
        break;  
    }  
}
```

```
    usleep(50000);  
}
```

```
gpioTerminate();  
return 0;  
}
```

```
|
```